Ders 17 Genişletilmiş Notu: Heap İşlemleri, Öncelikli Kuyruk ve Huffman Kodlaması

Ders 17 Genişletilmiş Notu: Heap İşlemleri, Öncelikli Kuyruk ve Huffman Kodlaması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Heap Ağacının Tekrarı
- 2. Heap Ağacına Eleman Ekleme: Sift Up
- 3. Kökü Silme: Sift Down
- 4. Çocuk Sayısını Anlamak ve Sift Down Kodu
- 5. Heap Sort Tekrarı
- 6. Sıralama Algoritmalarının Karşılaştırılması
- 7. Heap Sort Neden $O(n \log n)$?
- 8. Hazır Fonksiyonları Kullanmak
- 9. Öncelikli Kuyruk (Priority Queue)
- 10. Huffman Kodlaması
- 11. Huffman Ağacının Öncelikli Kuyruk Mantığıyla Kurulması
- 12. Öncelikli Kuyruğa Dönüş: Hizmet Etme ve Yapı Seçimi
- 13. Sınavdaki Algoritmik Soruların Çözüm Mantığı

1. Heap Ağacının Tekrarı

Hoca, araya zaman girdiği için geçen derste konuşulan heap ağacını (heap tree) önce slayt üzerinden hatırlattı. Heap ağacı genel olarak ikili ağaç (binary tree) yapısında kullanılır. Üçlü ağaç biçiminde de olabilir; sınavda bu yüzden üçlü bir örnek sorulmuştu. Üç özelliği vardır:

- **Sola dayalıdır:** Yeni eleman ilk boş göze konur. Dizide saklanınca arada boş göz kalmaz.
- **Dengelidir (balanced):** n eleman varsa yüksekliği $\log n$ olur.
- **Heap özelliği vardır:** Max heap'te ebeveyn (parent) çocuklarından (child) büyüktür. Min heap'te ebeveyn çocuklarından küçüktür.

Hocanın örneği bir max heap'ti: kökte 20, çocuklarında 16 ve 12 vardı. 20 hem 16'dan hem 12'den büyüktür. Ama 16 ile 12 arasında bir ilişki yoktur.

> **Hocanın vurgusu:** Heap ağacında sol ve sağ çocuk arasında bir kural yoktur. İkili arama ağacındaki (binary search tree) "küçük solda, büyük sağda" gibi bir bağlantı burada aranmaz.

1.1 Heap Neden Dizide Saklanır?

Normalde ağaçları bağlı listede (linked list) saklamak daha avantajlı olabilir. Çünkü ağaçta denge yoksa, dizide saklandığında çok sayıda boş göz kalır. Heap ağacı ise zaten dizide saklanmak üzere tasarlanmıştır. Sola dayalı ve dengeli olması, arada boş göz kalmaması demektir.

Dizide i. gözdeki düğümün çocukları ve ebeveyni şu gözlerdedir:

$$\text{sol çocuk} = 2i + 1, \qquad \text{sağ çocuk} = 2i + 2, \qquad \text{ebeveyn} = \frac{i-1}{2} \text{ (tam sayı bölmesi)}$$

Hoca ebeveyn formülünü iki sayıyla sınadı. Sıfırıncı göz kökse birinci ve ikinci gözler onun çocuklarıdır. Üçüncü gözdeki düğümün çocukları da yedinci ve sekizinci gözlerdir. Geriye doğru hesaplarsak:

- 8. gözdeki elemanın ebeveyni: $(8-1)/2 = 7/2 = 3$.
- 7. gözdeki elemanın ebeveyni: $(7-1)/2 = 6/2 = 3$.

İki sonuç da 3. gözü verir, yani formül doğrudur.

2. Heap Ağacına Eleman Ekleme: Sift Up

2.1 Fikir ve Örnek

Elimizde n elemanlı bir heap varsa elemanlar 0'dan $n-1$. göze kadar yerleşmiştir. Yeni gelen eleman ilk boş göze, yani `Heap[n]`'e konur. Bu sırada ağacın şekli korunur, ama heap özelliği bozulmuş olabilir. Bu yüzden eleman doğru yerine taşınmalıdır. Bu işleme **sift up** denir.

Slayttaki adımlar kısa nottaki kareden şöyledir:

1. Yeni elemanı bir sonraki boş düğüme koy.
2. Yeni eleman ebeveyninden küçük veya eşit olana kadar (ya da kök olana kadar) yukarı taşı.

Hoca, slayttaki "küçük eşit" ifadesinin min heap için olduğunu söyledi. Örneğimiz max heap olduğu için eleman ebeveyninden büyük olduğu sürece yukarı taşınır.

> **Ders sonrası not:** Hoca "küçük eşit" ifadesini min heap'e bağladı; ama kısa nottaki slayt metni büyük olasılıkla "küçük veya eşit olana kadar" biçimindeydi. Bu kalıp max heap'in durma şartıdır: eleman ebeveyninden küçük veya eşit olunca durulur. Hocanın uyguladığı kural (büyük olduğu sürece yukarı taşı) ile aynı şeydir.

Örnek. Max heap `[20, 16, 12, 5, 8]`, $n = 5$. Yeni eleman 35.

```
35, 5. göze konur           : [20, 16, 12, 5, 8, 35]
ebeveyn (5-1)/2 = 2 -> 12   : 12 < 35, yer değiştir -> [20, 16, 35, 5, 8, 12]
ebeveyn (2-1)/2 = 0 -> 20   : 20 < 35, yer değiştir -> [35, 16, 20, 5, 8, 12]
adr = 0 (kök), dur
```

Bu işlem boyunca ağacın geri kalanına hiç bakılmadı. Yalnızca 35'in ebeveyn yolu incelendi. Ağacın solu ve sağı birbirinden bağımsız hareket eder.

> **Hocanın vurgusu:** Sift up sırasında yalnızca yeni elemanın ebeveyn yoluna bakılır. Geri kalan kısım bizi ilgilendirmez.

2.2 Durma Şartları

Yukarı taşımanın iki durma şartı vardır:

- **Eleman yerine yerleşti:** Hoca bunu üstte 85 olan bir örnekle anlattı. 35 ile 85 karşılaştırılır. 35 küçük olduğu için burada durulur, daha yukarı gidilmez.
- **Köke gelindi:** 35 örneğinde olduğu gibi, eleman herkesten büyükse köke gelince durulur. Çünkü yukarıda gidecek yer yoktur.

2.3 Kod

Kod, yeni elemanın bulunduğu indisi `adr` ile, elemanı `item` ile, diziyi `Heap` ile gösterir. Köke gelme şartı `adr > 0`, yerine yerleşme şartı da bir bayraktır (flag). Hoca bayrakla yazmayı daha anlaşılır bulduğunu, ama isteyenin başka türlü de yazabileceğini söyledi.

Kod slayttan alınmıştır. Slayttaki parçada görünmeyen `parent` bildirimi, `swap` fonksiyonu ve başlık satırı derlenebilmesi için eklendi.

```c
#include <stdio.h>

void siftDown(int Heap[], int n);

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

void siftUp(int item, int Heap[], int adr)
{
    int parent, stop = 0;
    while ((adr > 0) && (!stop))
    {
        parent = (adr - 1) / 2;
        if (Heap[parent] < item)
        {
            swap(&Heap[adr], &Heap[parent]);
            adr = parent;
        }
        else
            stop = 1;
    }
}
```

Kodu 35 örneğinde izleyelim. 35 beşinci gözdedir. Ebeveyn $(5-1)/2 = 2$. gözdedir ve orada 12 vardır. `Heap[2] < 35` olduğu için yer değiştirilir. 35 artık yukarı taşındığı için yeni adresi ebeveynin adresidir: `adr = 2`. Sonra ebeveyn $(2-1)/2 = 0$. gözdür ve orada 20 vardır. 20 de küçüktür, yer değiştirilir, `adr = 0` olur. Döngü başına dönüldüğünde `adr > 0` sağlanmaz ve çıkılır. 35, 12'den küçük olsaydı `stop = 1` olur, döngü yine biterdi.

Ekleme fonksiyonu yeni elemanı `n`. göze yazar, `siftUp`'ı çağırır ve eleman sayısını bir artırır:

```c
void insert(int item, int Heap[], int *n)
{
    Heap[*n] = item;
    siftUp(item, Heap, *n);
    (*n)++;
}
```

2.4 Karmaşıklık

En iyi durumda yeni eleman hiç yer değiştirmez. Hoca 35 eklendikten sonraki ağaca 3 eklemeyi önerdi. `[35, 16, 20, 5, 8, 12]` dizisinde 3, 6. göze gelir ve ebeveyni $(6-1)/2 = 2$. gözdeki 20'dir (ara adım eklendi). 3 ile 20 karşılaştırılır, 20 büyük olduğu için 3 yerinde kalır. En kötü durumda ise 35 gibi köke kadar çıkar. Bu durumda ağacın diğer kısmına hiç bakılmaz, yalnız seviye sayısı kadar işlem yapılır. Ağaç dengeli olduğu için yüksekliği $\log n$'dir. Dolayısıyla ekleme en kötü durumda $O(\log n)$'dir.

> **Ek bilgi (derste anlatılmadı):** Yükseklik sonucu şöyle gösterilir. h yüksekliğindeki sola dayalı dengeli ağaçta son seviyenin en az bir düğümü vardır, üst seviyeler de doludur. Bu yüzden $2^h \leq n \leq 2^{h+1} - 1$ olur. Buradan $h = \lfloor \log_2 n \rfloor$ çıkar.

3. Kökü Silme: Sift Down

3.1 Neden Gerekli ve Nasıl Yapılır?

Kökü silme hem heap sort'ta hem de birazdan göreceğimiz öncelikli kuyrukta gerekir. Hoca önce yapılmaması gerekeni söyledi: kökü adım adım aşağı taşımaya, her seferinde bir alttakini yukarı almaya çalışmak zor ve karmaşık bir yoldur.

> **Hocanın vurgusu:** Kökü silmenin kolay yolu, onu en sondaki elemanla yer değiştirmektir. Sonra son eleman yokmuş gibi davranılır.

Kök son göze gelince artık `n-1`. gözdedir. Dizi boyutu bir azaltılırsa o eleman görülmez. İsterseniz kullanırsınız (heap sort'ta olduğu gibi), kullanmayacaksanız hiç ilgilenmezsiniz.

Ama köke gelen eleman heap özelliğini bozmuş olabilir. Onu doğru yere taşımak gerekir. Bu işlem **sift down**'dır. Ağaca yeniden heap özelliğini kazandırmaya heapify de denir. Sift up'ta yeni eleman aşağıda olduğu için yukarı taşınıyordu. Burada eleman aşağıdan alınıp köke konduğu için aşağı taşınır.

Sift down iki aşamalıdır, çünkü iki çocuk arasında bir ilişki yoktur; hangisinin büyük olduğu önceden bilinemez:

1. İki çocuk birbiriyle karşılaştırılır, büyüğü seçilir.
2. Seçilen çocuk ebeveynle karşılaştırılır. Çocuk büyükse yer değiştirilir.

Örnek. Max heap `[28, 20, 12, 16, 8, 5]`, kök 28 siliniyor.

```
28 saklanır, son eleman 5 köke gelir, n = 5 : [5, 20, 12, 16, 8]
5'in çocukları 20 ve 12; büyüğü 20; 20 > 5  : [20, 5, 12, 16, 8]
5'in çocukları 16 ve 8;  büyüğü 16; 16 > 5  : [20, 16, 12, 5, 8]
5 yaprak oldu, dur
```

Durma şartları yine ikidir:

- **Eleman yerine yerleşti:** Hocanın örneğinde 5'in çocukları 3 ve 1 olsaydı, 3 büyük çıkar, 5 ile 3 karşılaştırılır. 5 büyük olduğu için durulur.
- **Eleman yaprağa geldi:** Aşağıda kimse kalmamıştır.

Soru: 5'i silmediğimizde ağacın dengesi bozulmaz mı?

Cevap: 5 silinmiyor, silinen 28. 5 yalnız 28 ile yer değiştirdiği için heap özelliğini bozdu. Şimdi 5 yerine yerleştiriliyor, yani ağaç heapify ediliyor.

3.2 `removeMax` Kodu

Kök en büyük elemansa işlem `removeMax`, en küçük elemansa `removeMin` olur. Bu tamamen ağacın min heap ya da max heap oluşuna bağlıdır. Hangisinin seçileceği de ihtiyaca bağlıdır: bazen en büyük elemanın kökte olması istenir, bazen en küçüğün.

Silinen değer kullanılabilsin diye geri döndürülür. Hoca, bunun yerine başka şeyler de yapılabileceğini söyledi. Kod slayttan alınmıştır:

```
int removeMax(int Heap[], int *n)
{
    int retValue = Heap[0];
    Heap[0] = Heap[*n - 1];
    (*n)--;
    siftDown(Heap, *n);
    return retValue;
}
```

Burada son eleman köke kopyalanır ve `n` bir azaltılır. Diziden bakınca o göz artık yoktur: `[5, 20, 12, 16, 8]`. Sonra `siftDown` ile ağaç heapify edilir.

3.3 Sol Çocuk Hep Büyük mü?

Soru: Burada sol kardeş hep sağdakinden büyük. Yapı böyle mi olmak zorunda?

Cevap: Hayır, öyle denk geldi. Hoca bunu göstermek için tahtada yeni bir ağaç çizdi. Kökte 5 vardı. Üst seviyede 13 ile 8 karşılaştırıldı, 13 büyüktü ve 5 ile yer değiştirdi. Alt seviyede ise çocuklar 4 ve 9'du. 9 sağdaydı ve büyük olan oydu; 5 ile 9 yer değiştirdi. Büyük çocuk sağda da olabilir.

> **Hocanın vurgusu:** Önemli olan, önce iki çocuğu karşılaştırmak, sonra büyük olanı ebeveynle karşılaştırmaktır. Çocuk ebeveynden büyükse yer değiştirilir.

4. Çocuk Sayısını Anlamak ve Sift Down Kodu

4.1 Üç Durum

Şimdiye kadarki örneklerde her düğümün iki çocuğu vardı, yani her şey çok şıktı. Oysa her zaman öyle olmaz. Hoca kökte 5, çocuklarında 11 ve 17 olan bir örnek çizdi. 17 büyüktür ve 5 ile yer değiştirir. Ama 5 yeni yerine indiğinde altında tek çocuk vardır. O zaman iki çocuğu karşılaştırma adımı uygulanamaz. Demek ki önce kaç çocuk olduğunu anlamak gerekir.

Bu, eleman sayısı n'ye bakılarak anlaşılır. n elemanlı dizide son eleman $n-1$. gözdedir, n. göz yoktur. Bu yüzden bir gözün var olması için indisinin n'den küçük olması gerekir.

- **Durum 1, iki çocuk:** $2i+2 < n$ ise sağ çocuk vardır, yani iki çocuk vardır.
- **Durum 2, tek çocuk:** $2i+2 < n$ değil ama $2i+1 < n$ ise yalnız sol çocuk vardır.
- **Durum 3, yaprak:** İkisi de sağlanmıyorsa düğüm yapraktır.

Kontrol sağ çocuktan başlar. Hoca bir yerde $2i+1$'den başlayınca hemen özür dileyip $2i+2$'den başlamak gerektiğini söyledi. Dersteki örneklerin hepsi şu tabloda toplanmıştır:

n	i	$2i+2 < n$?	$2i+1 < n$?	Sonuç
3	0	$2 < 3$ evet	gerekmez	İki çocuk
2	0	$2 < 2$ hayır	$1 < 2$ evet	Tek çocuk
4	1	$4 < 4$ hayır	$3 < 4$ evet	Tek çocuk
3	1	$4 < 3$ hayır	$3 < 3$ hayır	Yaprak
6	3	$8 < 6$ hayır	$7 < 6$ hayır	Yaprak

Örnek. Hoca bu testi tek bir silme adımında birlikte gösterdi. Dizi `[5, 41, 17, 4]`, $n = 4$, 5 kökte.

1. $i = 0$: $2 < 4$, iki çocuk var. 41 ile 17 karşılaştırılır, 41 büyük. 41 ile 5 karşılaştırılır, 41 büyük. Yer değiştirilir: `[41, 5, 17, 4]`.
2. $i = 1$: $4 < 4$ değil, iki çocuk yok. $3 < 4$, tek çocuk var. 5 ile 4 karşılaştırılır. 5 büyük olduğu için yerinde kalır.

4.2 Büyük Çocuğu Bulan Fonksiyon

Hoca, moladan önce "karşılaştırmayı kimle yapacağım?" diye sordu. Bir öğrenci "küçük çocukla" deyince önce bunu onayladı.

> **Ders sonrası not:** Hoca "küçük çocukla" dedi; doğrusu büyük çocuktur, çünkü örnek max heap'tir. Hoca bunu ileride kendisi de düzeltti. Slayttaki fonksiyonun adı `findSmallestChild`'dır, ama max heap için büyük çocuğun indisini döndürür. Hocanın deyişiyle "isim yanlış, olay doğru".

Fonksiyon bir değer değil adres döndürür. Hoca burada adres derken elemanın bulunduğu indisi kastetti.

Soru: Adres dediğimiz indis değil mi, neticede dizilerle yapıyoruz?

Cevap: Evet, indis. Adres de denebilir, indis de; ikisi de doğru.

Slayt koduyla aynı mantık aşağıdadır. Durum sırası hocanın anlatımındaki gibidir. Kısa nottaki adımlardan koda dökülmüştür.

```
/* Slayttaki ad korunmuştur; max heap için BÜYÜK çocuğun indisini döndürür. */
int findSmallestChild(int Heap[], int i, int n)
{
    int child1 = 2 * i + 1;
    int child2 = 2 * i + 2;
    if (child2 < n)
    {
        if (Heap[child2] > Heap[child1])
            return child2;
        return child1;
```

```
    }
    if (child1 < n)
        return child1;
    return -1;
}
```

`-1`, "karşılaştıracak kimse yok" anlamındadır, çünkü yapraktayız.

4.3 `siftDown` Kodu

Döngünün iki durma durumu vardır: yapraktaysak inecek yer yoktur (`child == -1`), ya da altımızdakiler bizden küçüktür. Kod slayttan alınmıştır:

```
void siftDown(int Heap[], int n)
{
    int parent = 0, child;
    child = findSmallestChild(Heap, parent, n);
    while ((child != -1) && (Heap[child] > Heap[parent]))
    {
        swap(&Heap[child], &Heap[parent]);
        parent = child;
        child = findSmallestChild(Heap, parent, n);
    }
}
```

Hoca bunu ebeveyni büyük olan bir örnekle izledi. Kökte 20, çocuklarında 18 ve 19 var. 18 ile 19 karşılaştırılır, 19 büyüktür ve adresi döner. Sonra 20 ile 19 karşılaştırılır. Ebeveyn büyük olduğu için hareket etmeye gerek yoktur.

28 örneğinde ise 20 ile 12 karşılaştırılır, sonra 20 ile 5. Çocuk ebeveynden büyük olduğu için 20 yukarı, 5 aşağı gider. 5'in yeni yeri çocuğun eski yeridir (`parent = child`). Yola buradan devam edilir.

Karmaşıklık, eklemedeki gibidir. 5'i aşağı taşımanın en kötü durumu onu en aşağıya taşımaktır. Ağacın yüksekliği $\log n$ olduğu için sift up da sift down da en kötü durumda $O(\log n)$'dir.

5. Heap Sort Tekrarı

5.1 Mantık

Max heap'te en büyük eleman kökte durur. Kök son elemanla yer değiştirilirse en büyük eleman dizinin sonuna gider. Dizi sıralı olsaydı olacağı yer de orasıdır. Sonra köke gelen eleman sift down ile aşağı taşınır ve bakılan bölge bir küçültülür. Böylece yerleşmiş elemanlara dokunulmaz, dizi yavaş yavaş sıralanır.

Hoca döngüyü tahtada yazdı. i, $n-1$'den başlar ve her adımda bir azalır. Kısa nottaki satırdan tam fonksiyon haline getirilmiştir:

```
void heapSort(int Heap[], int n)
{
    int i;
    for (i = n - 1; i > 0; i--)
    {
        swap(&Heap[0], &Heap[i]);
        siftDown(Heap, i);
    }
}
```

> **Ek bilgi (derste anlatılmadı):** Bu döngü, dizi zaten heap iken çalışır. Derste de başlangıçtaki dizi "artık heap olmuş" kabul edildi. Sırasız bir dizi önce heap'e çevrilmelidir. Bu çeviri de en fazla $O(n \log n)$ sürer, yani toplam karmaşıklığı değiştirmez.

5.2 Örnek 1

Max heap `[20, 10, 13, 8, 7, 11, 12]`, $n = 7$.

1. 20 ile 12 yer değiştirir. 20 en büyük olduğu için doğru yerdedir. 12 köke gelir. İki çocuğu 10 ve 13'tür, 13 büyüktür. 13 ile 12 karşılaştırılır, 13 büyüktür, yer değiştirirler. Sonra 12'nin tek çocuğu 11'dir. 12 büyük olduğu için durur.
2. 13 ile 11 yer değiştirir. 11 köke gelir. 10 ile 12 karşılaştırılır, 12 büyüktür. 12 ile 11 karşılaştırılır, 12 büyüktür, yer değiştirirler. 11 artık yapraktır, çünkü altındaki göz bölgede değildir.

Hoca örneği burada, 12'yi sona taşıyacağını söyleyerek bıraktı. Tablodaki son dört satır bu yüzden aynı kuralla tamamlandı (ara adım eklendi).

i	Değişim	Sift down sonrası heap kısmı	Yerleşmiş kısım
6	20 ile 12	`13, 10, 12, 8, 7, 11`	`20`
5	13 ile 11	`12, 10, 11, 8, 7`	`13, 20`
4	12 ile 7	`11, 10, 7, 8`	`12, 13, 20`
3	11 ile 8	`10, 8, 7`	`11, 12, 13, 20`
2	10 ile 7	`8, 7`	`10, 11, 12, 13, 20`
1	8 ile 7	`7`	`8, 10, 11, 12, 13, 20`

Sonuç: `7, 8, 10, 11, 12, 13, 20`.

Hoca bu arada slayttaki bir diziye de kısaca değindi: 7 ile 15 yer değiştiriyor, bir sonraki adımda 0. göz ile 3. göz, sonra 0. göz ile 2. göz.

5.3 Örnek 2

Bir öğrenci tekrar anlatılmasını isteyince hoca küçük bir örnek yaptı. Max heap `[20, 10, 8, 7]`, $n = 4$.

$i = 3$**:** 0. ve 3. gözler değişir: `[7, 10, 8, 20]`. 20 yerine yerleşti. Şimdi bakılan bölge üç elemanlıdır, $n = 3$.

- 7 kökte: $2 \cdot 0 + 2 = 2 < 3$, iki çocuk var. 10 ile 8 karşılaştırılır, 10 büyük. 10 ile 7 karşılaştırılır, 10 büyük. Yer değiştirir: `[10, 7, 8, 20]`.
- 7 birinci gözde: $4 < 3$ değil. $3 < 3$ değil. 7 yapraktır ve orada kalır.

$i = 2$**:** 0. ve 2. gözler değişir: `[8, 7, 10, 20]`. Artık $n = 2$.

- 8 kökte: $2 < 2$ değil, iki çocuk yok. $1 < 2$, tek çocuk var. 8 ile 7 karşılaştırılır, 8 büyük, yerinde kalır.

$i = 1$**:** 0. ve 1. gözler değişir: `[7, 8, 10, 20]`. Artık $n = 1$.

- 7 kökte: $2 < 1$ değil, $1 < 1$ değil. 7 yapraktır.

Sonuç: `7, 8, 10, 20`.

Hoca bu izlemede n değerini bir iki kez yanlış söyleyip hemen düzeltti. Önemli olan, `siftDown`'a n yerine o anki i değerinin gitmesidir. 1. gözdeki 7'nin altındaki 3. gözde 20 duruyor gibi görünür, ama orası artık bölgede değildir.

> **Hocanın vurgusu:** i her seferinde bir azaldığı için dizinin bakılan bölümü de her seferinde bir azalır. Böylece yerleşmiş elemanlara dokunulmaz.

Dersteki fonksiyonların tamamı aşağıdaki `main` ile birleştirilip `gcc -Wall` ile uyarısız derlendi. `main` ve `printArray` denemek için eklendi, örnekler derstekilerdir.

```c
void printArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
```

```
    printf("\n");
}

int main(void)
{
    int h1[10] = {20, 16, 12, 5, 8};
    int n1 = 5;
    int h2[10] = {28, 20, 12, 16, 8, 5};
    int n2 = 6;
    int h3[7] = {20, 10, 13, 8, 7, 11, 12};
    int h4[4] = {20, 10, 8, 7};
    int silinen;

    insert(35, h1, &n1);
    printArray(h1, n1);

    silinen = removeMax(h2, &n2);
    printf("silinen: %d, heap: ", silinen);
    printArray(h2, n2);

    heapSort(h3, 7);
    printArray(h3, 7);

    heapSort(h4, 4);
    printArray(h4, 4);
    return 0;
}
```

Çıktı:

```
35 16 20 5 8 12
silinen: 28, heap: 20 16 12 5 8
7 8 10 11 12 13 20
7 8 10 20
```

6. Sıralama Algoritmalarının Karşılaştırılması

Hoca bilinen sıralama algoritmalarını slayttaki tabloyla karşılaştırdı. Tablo kısa nottaki kareden alınmıştır.

Algoritma	En iyi	Ortalama	En kötü	Ek bellek
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^{1.5})$	$O(1)$
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(1)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Hocanın yorumları şunlardır:

- Selection sort ve insertion sort yalnız bir değişken kadar ek bellek kullanır. Insertion sort'un en iyi durumu dizinin sıralı olduğu durumdur.
- Quicksort ve merge sort ilerleyen derslerde görülecek. Merge sort yedek bir dizi kullandığı için n kadar ek belleğe ihtiyaç duyar. Quicksort'un en kötü durumu dizi sıralıyken ortaya çıkar.
- Heap sort ile merge sort'un en iyi, ortalama ve en kötü durumları aynıdır. Heap sort'un avantajı ek bellek kullanmamasıdır.

Hangisini seçeceğimiz soruldu. Dizi küçükse ya da "sıralı gibi" ise insertion sort tercih edilir. "Sıralı gibi" şu demektir: Dizi aslında sıralıdır, ama eklemeler ve çıkarmalar yüzünden arada sıra bozulur, tamamen bozulmaz. Büyük ve sıralı olmadığı bilinen dizilerde ise quicksort seçilir.

> **Ders sonrası not:** Tablodaki bazı satırlar ek açıklama ister. Hiç yer değiştirme olmayan turda duran (bayraklı) bubble sort, sıralı dizide $O(n)$'de biter. Quicksort'un ek belleği özyineleme yığını yüzünden ortalamada $O(\log n)$, en kötü durumda $O(n)$'dir. Quicksort'un sıralı dizide $O(n^2)$ olması, pivot olarak ilk ya da son elemanın seçildiği uygulamalar içindir. Shell sort'un değerleri ise kullanılan aralık (gap) dizisine bağlıdır.

7. Heap Sort Neden $O(n \log n)$?

Bir elemanı yukarıdan aşağı taşımanın (sift down) karmaşıklığı $\log n$'dir. Bu işlem n defa yapılır. Her seferinde bir takas yapılır, yapraktan bir eleman köke gelir ve sift down ile uygun yere iner. Toplam:

$$n \cdot \log n = O(n \log n)$$

Soru: Bu karmaşıklık ağaç yapısı kabul edilerek mi bulunuyor? Yani kökte maksimum ya da minimum olduğu kabul edilince mi ondan sonraki aşama $n \log n$ oluyor?

Cevap: Kesinlikle. Heap ağacının şartı budur. Bütün ebeveynler çocuklarından büyükse köktekinin herkesten büyük olması gerekir. En aşağı taşındığında da dizi sıralı olsaydı bulunacağı yere gitmiş olur. Ayrıca ağaç dengeli olduğu için yükseklik en kötü durumda $\log n$'dir. Bir elemanı $\log n$ adımda taşıyıp bunu n defa yapıyorsak karmaşıklık $n \log n$'dir.

Soru: Gösterimde bazen $\log_2 n$, bazen $\log n$ yazıyorsunuz. Fark var mı?

Cevap: Yok. Karmaşıklıkta taban yazılmazsa 2 kabul edilir. Kitaplarda da genelde yazılmaz. Üç ya da beş tabanı kastediliyorsa mutlaka belirtilmelidir.

> **Ders sonrası not:** Taban, Big-O sonucunu zaten değiştirmez, çünkü farklı tabanlar arasında yalnız sabit bir çarpan vardır: $\log_2 n = \log_{10} n / \log_{10} 2$. Bu çarpan yaklaşık $3{,}32$'dir.

8. Hazır Fonksiyonları Kullanmak

Dersin bu bölümünde sınav üzerine konuşuldu. Konunun akademik yanı, "hazır kabul edilen fonksiyon" kavramının ne anlama geldiğidir. Sınavdaki kuyruk sorusunda `enqueue`, `dequeue` ve `isEmpty` fonksiyonlarının yazılmaması, hazır olduğunun varsayılması istenmişti.

Soru: Aynı işi yapan fonksiyonu herkes farklı yazar. Biri tek, biri iki parametreyle yazabilir. Fonksiyonun nasıl uygulandığını bilmeden çözüm nasıl üretilir?

Cevap: Hoca bunu `printf` benzetmesiyle cevapladı. `printf` fonksiyonunu yazmıyorsunuz, kullanıyorsunuz. `scanf` için de aynısı geçerlidir. Burada da aynı mantık kullanılır:

- `enqueue` kuyruğa eleman ekler. Demek ki bir kuyruğa ve bir elemana ihtiyacı vardır.
- `dequeue` kuyruğun başındakini çıkarıp size verir. Parametreyle mi, `return` ile mi verdiği önemli değildir. Önemli olan çıkan elemanın elde edilebilmesidir.
- `isEmpty` kuyrukta eleman olup olmadığına bakar.

Hazır kabul edilme nedeni de açıktır. Bu sorunun hedefi kuyruk fonksiyonlarını yazdırmak değil, kuyruğu kullandırmaktı. Gerçek dünyada da bir kuyruk fonksiyonu varken kimse sizden onu yeniden yazmanızı istemez. Asıl yazılacak kısım, örneğin yazıcıya gelen isteklerin kuyruktan çıkarılıp işlenmesidir. Özellikle uygulama biçimi de verilmedi, çünkü isteyen dairesel kuyruk, isteyen bağlı liste kullanabilsin istendi.

Hoca aynı mantığı başka örneklerle genişletti. Selection sort ya da insertion sort'u olduğu gibi yazdırmak, parantezlerin doğru sıralanıp sıralanmadığı sorusunda yığının `pop` fonksiyonunu yazdırmak bir şey ölçmez. Anlatılmış ve bilinen bir şeyi yeniden istemek öğrenciyi ezbere iter.

Hocanın vurgusu: Ezberlemeyin, düşünmeye alışın. Algoritmik tasarım tamamen analitik düşüncedir. Çok farklı soru çözdükçe, derste anlatılanları bakmadan kendi başınıza çözmeye çalıştıkça sorular kolay görünmeye başlar. Ayrıntıdan çok genel tasarıma odaklanın.

9. Öncelikli Kuyruk (Priority Queue)

9.1 Neden Gerekli?

Kuyruk yapısında gelen eleman sona eklenir ve en öndeki işe hizmet edilir. Bir eleman çıkınca, bağlı listede sıradaki öne geçer; dairesel yapıda ise elemanlar bittikçe başa doğru yerleşir. Ama bazen sıraya uymak istemeyiz. Hoca üç örnek verdi:

- **Acil servis:** Sırayla hastalara bakılırken kanaması çok fazla bir hasta gelir. Bu kişi sıranın önüne alınmalıdır.
- **İşletim sistemi:** Kuyrukta sıra çok uzun bir işte, arkasında ise bir sürü küçük iş var. Uzun işe hizmet edilirse küçük işler çok uzun süre bekler. İşleri paylaştırarak yapma şansımız yoksa küçük iş öne alınır, uzun iş bekletilir.
- **Hastane:** 80 yaş üstü hastalar genelde kuyruğun önüne alınır.

9.2 Üç Yol

Elemanların önceliği olduğunda üç seçenek vardır. Örneklerde sayılar önceliği gösterir ve en öncelikli eleman en küçük sayıdır.

1. Sırasız yapı. Gelen eleman kuyruğa, sıraya yerleştirilir. Hizmet verirken bütün diziye bakılıp en öncelikli iş bulunur. Yerleştirme bir adım, hizmet n adımdır. Örneğin `[5, 2, 7, 1]` dizisine 9 gelirse sona konur, ama en küçük her seferinde aranır.

2. Sıralı yapı. Gelen eleman, kuyruğa insertion sort mantığıyla sırayı bozmadan yerleştirilir. En küçük hep başta olduğu için hizmet bir adımdır. Yerleştirirken n adım kaybedilir.

Örnek: `[2, 3, 4, 9]` dizisine 7 geliyor. 7, 2'den küçük mü? Hayır. 3'ten? Hayır. 4'ten? Hayır. 9'dan? Evet. 9 bir göz kaydırılır, 7 araya girer: `[2, 3, 4, 7, 9]`. Önce yer bulunur, sonra yer açılır. Hoca örnek olarak 1 yerine 7'yi seçti, çünkü 1 hemen başa yerleşir ve bu en iyi durumdur.

Bu yapı dairesel (circular) bir diziyse çıkarmada hiçbir şey kaydırmak gerekmez, bu yüzden çıkarma bir adımdır. Bağlı liste olsaydı yine yerleştirme için tek tek bakılır. Çıkarmada ise başı koparmak yeter: `2, 3, 4, 9` listesinde 2'nin bağlantısı kesilir, liste başı 3 olur.

3. Binary heap. Heap ağacında kökte her zaman en küçük ya da en büyük değer vardır. Bu yüzden öncelikli kuyruk genellikle heap ağacı kullanır. Hizmet edilecek en öncelikli iş zaten köktedir.

- **Ekleme:** Yeni eleman ağaca yerleştirilir ve sift up yapılır, $O(\log n)$.
- **Çıkarma:** Kök alınır, son elemanla yer değiştirilir ve heapify edilir. Bir adımda kök alınır ama ağacı düzeltmek gerekir, $O(\log n)$.
- **En öncelikliyi bulma:** Kökte durur, $O(1)$.

Soru: En küçük elemanı ağaçtan çıkarmanın karmaşıklığı sabit mi?

Cevap: Hayır, $\log n$. Elemanı bir adımda çıkarırsınız, ama ağacı heapify etmeniz gerekir.

Min heap'te hoca kökte 3 olan bir örnek çizdi. 3 çıkınca ağacın `5, 7, 4, 9` haline geleceğini söyledi. Bir öğrenci "4 yukarı çıkmayacak mı, minimum ağaç yapmıyor muyuz?" diye itiraz etti. Hoca haklı olduğunu söyledi ve 4'ü 14 yaptı. Düzeltilmiş hali `[3, 5, 14, 7, 9]` dizisiyle şöyle çalışır (ara adım eklendi):

```
3 çıkar, son eleman 9 köke gelir : [9, 5, 14, 7]
9'un çocukları 5 ve 14; küçüğü 5 : [5, 9, 14, 7]
9'un tek çocuğu 7; 7 < 9         : [5, 7, 14, 9]
```

9.3 Slayttaki Ekleme ve Çıkarma İzi

Hoca slaytta bir min heap üzerinde ekleme ve çıkarmayı karşılaştırmalarla izledi.

- **5'i eklemek:** 5 en sona konur. 5 ile 46 karşılaştırılır, 5 yukarı çıkar, 46 iner. 5 ile 26 karşılaştırılır, 5 yukarı çıkar. 5 ile 9 karşılaştırılır, 5 yukarı çıkar. 5 yerine yerleşir.
- **Minimumu çıkarmak:** Minimum kökte olduğu için onu bulmak bir adımdır. Son eleman 46 köke alınır ve aşağı taşınır. 5 ile 4 karşılaştırılır, 4 küçüktür; 46 ile 4 yer değiştirir. 18 ile 20 karşılaştırılır, 18 küçüktür; 46 ile 18 yer değiştirir. 30 ile 19 karşılaştırılır, 19 küçüktür; 46 ile 19 yer değiştirir.

Kısa nottaki slayt metnine göre çıkarma adımları şöyledir:

1. Kökteki değeri döndür.
2. Son elemanı köke taşı.
3. Yeni kökü, çocuklarından küçük veya eşit olana kadar (ya da yaprak olana kadar) aşağı taşı.

   Ders sonrası not: Kısa nota göre slaytta bu adımda büyük olasılıkla "çocuklarından büyük veya eşit olana kadar" yazıyordu. Min heap için şart tersidir: yeni kök, çocuklarından küçük veya eşit olana kadar aşağı taşınır.

9.4 Karşılaştırma Tablosu

Kısa nottaki slayt tablosu en kötü durum maliyetlerini verir:

Yapı	Ekleme	En büyüğü/küçüğü çıkar	En büyüğü/küçüğü bul
Sıralı dizi	N	1	1
Sıralı liste	N	1	1
Sırasız dizi	1	N	N
Sırasız liste	1	N	N
Binary heap	$\log N$	$\log N$	1

10. Huffman Kodlaması

10.1 Fikir

Hoca Huffman kodlamasını (Huffman coding) "kullanım yeri şaşırtan" bir konu diye tanıttı. Bu konu ayrık matematik dersinden hatırlanıyordu.

Normalde bir yazıda her ASCII karakter 8 bit tutar. Oysa harflerin kullanımı aynı değildir. Hocanın örneği: Bin karakterlik bir yazıda 300 kez `a`, 100 kez `e` geçer, ama yalnız 3 tane yumuşak `g` vardır. O 3 harf için de birer bayt, 300 kez kullanılan `a` için de birer bayt harcanır.

Huffman kodlaması 1952'de David Huffman tarafından geliştirildi. Hoca, bunun bir dersteki proje ya da özgün çözüm ödevi olarak ortaya çıktığını anlattı.

> **Ders sonrası not:** Hoca bu dersin ekonomi dersi olduğunu söyledi; doğrusu büyük olasılıkla MIT'de Robert Fano'nun verdiği bilgi teorisi dersidir.

Fikir şudur: Çok kullanılan bilgi az bitle, az kullanılan bilgi çok bitle gösterilir. Az kullanılan sembol bir iki kez geçtiği için ona çok bit harcamak büyük kayıp değildir. Çok kullanılanı az bitle göstermek ise toplamda kazandırır.

10.2 Önek Kod (Prefix Code)

Kodların uzunluğu değişince bir sorun çıkar: Yan yana duran bitleri nasıl ayıracağız? `a`'yı `0`, `b`'yi `00` ile kodlarsak `000` dizisinde üç `a` mı var, yoksa bir `b` bir `a` mı, anlayamayız. Bu yüzden hiçbir kod başka bir kodun öneki olmamalıdır. Buna önek kod denir.

Kısa nottaki örnekte `0, 1, 11` önek kod değildir, çünkü `1`, `11`'in başıdır. `0, 11, 101` önek koddur.

Ders sonrası not: Hoca önek kodu anlatırken “B’nin içerisinde A geçemez” dedi; doğrusu B’nin **başında** A olamaz. Bir kod başka bir kodun ortasında ya da sonunda geçebilir. Nitekim `0, 11, 101` önek koddur, oysa `0`, `101`’in ortasında geçer. Hoca da az sonraki örnekte aynı noktaya geldi: `c`’nin kodunun içinde `0` geçer, ama bu önek kod olmasını bozmaz.

10.3 Örnek 1: Kodları Bulmak

`a`, `b`, `c` ve boşluktan oluşan bir yazı var. Burası bir resim de olabilirdi. Hoca, JPEG ve MPEG’in de bir aşamasında Huffman kodlaması kullandığını söyledi. Önce her sembolün kaç kez geçtiği bulunur:

Sembol	Frekans
boşluk	2
a	4
b	5
c	1

Toplam $2+4+5+1 = 12$ karakter vardır. Semboller frekansa göre sıralanır ve en küçüklerden başlanarak birleştirilir. Slaytta liste `b, a, boşluk, c` diye okunuyor. Önemli olan her adımda en küçük ikisinin alınmasıdır. Birleşen grup sırayı bozmayacak şekilde yerine konur.

```
c:1 + boşluk:2 -> 3
3 + a:4        -> 7
7 + b:5        -> 12 (kök)
```

Ağaç, bu birleştirmeler açılarak yazılır. `b` yapraktır ve yerine yerleşir. Öteki taraf açılır, `a` yerine yerleşir. Sonra `c` ve boşluk açılır. Sol kollara 0, sağ kollara 1 verilir. Bir sembolün kodu, kökten o yaprağa giden yol üzerindeki bitlerdir.

```
12
+--0-- b (5)
+--1-- 7
       +--0-- 3
       |      +--0-- c (1)
       |      +--1-- boşluk (2)
       +--1-- a (4)
```

```
Kodlar: b = 0, a = 11, c = 100, boşluk = 101
```

En çok geçen `b` bir bitle, ikinci çok geçen `a` iki bitle, öteki ikisi üçer bitle gösterildi.

10.4 Kazanç

Yazı normalde $12 \times 8 = 96$ bit tutar. Huffman ile:

$$5 \cdot 1 + 4 \cdot 2 + 1 \cdot 3 + 2 \cdot 3 = 5 + 8 + 3 + 6 = 22 \text{ bit}$$

96 bitlik yazı 22 bite iner.

10.5 Kodlama ve Kod Çözme

Hoca `b`, `a`, `c`, boşluk, `a` dizisini kodladı:

```
b    a    c     boşluk  a
0    11   100   101     11     ->  01110010111
```

Ders sonrası not: Hoca dizinin kodunu okurken “B’nin kodu 0, 0” dedi; doğrusu tek bit `0`’dır. Kod çözme sırasında kendisi de “sıfır gördüm, sola git, yaprak buldum” diyerek tek biti kullandı. `c`’yi çözerken de bir yerde “sıfır gördüm, sağa git” dedi; doğrusu `1` sağa, iki `0` sola gider.

Kod çözme (decode) ağaç üzerinde ilerleyerek yapılır. Kökten başlanır, `0` gelince sola, `1` gelince sağa gidilir. Yaprağa gelince o sembol yazılır ve köke dönülür.

Okunan bitler	Yol	Yazılan
`0`	sol, yaprak	`b`
`11`	sağ, sağ, yaprak	`a`
`100`	sağ, sol, sol, yaprak	`c`
`101`	sağ, sol, sağ, yaprak	boşluk
`11`	sağ, sağ, yaprak	`a`

Yaprağa gelindiğinde iş biter, çünkü hiçbir kod başka bir kodun devamı değildir. Bu yüzden belirsizlik çıkmaz.

10.6 Ağacın Boyutu ve Sıkıştırma

Soru: Toplam veri, sıkıştırılmış hal artı ağacın boyutu oluyor, değil mi?

Cevap: Evet, ağacın da boyutu vardır. 96 bit ile 22 bitten söz edince büyük görünür. Ama gerçekten büyük bir dosyada, örneğin bir megabaytlık bir resimde, ağacın boyutu o kadar yer tutmaz.

Hoca, sıkıştırmada asıl önemli olanın gereksiz (redundant) bilgiden kurtulmak olduğunu söyledi. Örneğin taranmış bir sayfada arka plan beyaz, yazılar siyahtır. Resmin üçte biri tek renk olsa ve o renk bir bitle gösterilse çok büyük bir yer kazanılır.

Soru: Böyle sıkıştırınca kayıp oluyor mu? Örneğin fotoğrafı JPEG yapınca?

Cevap: Huffman kayıpsızdır (lossless), burada hiçbir şey kaybedilmedi. JPEG'te kayıp olması Huffman yüzünden değildir. JPEG, frekans düzleminde bir dönüşüm yapar ve gözün algılayamayacağı şekilde renk azaltması uygular. JPEG'te kayıplı ya da kayıpsız seçeneği de vardır: Kayıpsız seçilirse renk azaltılmaz.

> **Ders sonrası not:** Altyazıda dönüşümün adı bozuk geçiyor ("bir script for ya transform"). Hoca büyük olasılıkla "discrete Fourier transform" dedi; JPEG standardı büyük olasılıkla ayrık kosinüs dönüşümü (discrete cosine transform) kullanır. Kaybın ana kaynağı da büyük olasılıkla bu dönüşümün katsayılarının kuantalanmasıdır.

11. Huffman Ağacının Öncelikli Kuyruk Mantığıyla Kurulması

Hoca bu kısma "çok heyecanlı bir nokta" diyerek başladı. Çünkü Huffman'ı görürken sorulması gereken bir soru vardır: İkisini toplayıp sonra sonucu öbürüyle topluyorsunuz, peki bunlar ağaca nasıl yerleşiyor? Ağaç bu hale nasıl geliyor? Cevap öncelikli kuyruk mantığıdır. Hoca, gerçekte de bunun öncelikli kuyrukla yapıldığını söyledi.

Algoritmanın adımları şunlardır:

1. Harfler, frekanslarına göre küçükten büyüğe bağlı listeye yerleştirilir.
2. En küçük iki düğüm alınır. Biri sol düğüm, biri sağ düğüm olur; toplamları köke yazılır. Böylece bir ikili ağaç elde edilir.
3. Yeni ağaç sırayı bozmayacak şekilde bağlı listeye eklenir.
4. Tek düğüm kalana kadar 2. ve 3. adımlar tekrarlanır.

Örnek. Beş farklı karakter var: `a`:3, `b`:3, `c`:1, `d`:1, `e`:2. Eşit frekanslılardan hangisinin önce geleceği fark etmez. Burada alfabe sırası kullanıldı.

```
Başlangıç               liste: c:1, d:1, e:2, a:3, b:3
c:1 + d:1 -> 2          liste: e:2, (c d):2, a:3, b:3
e:2 + (c d):2 -> 4      liste: a:3, b:3, (e (c d)):4
a:3 + b:3 -> 6          liste: (e (c d)):4, (a b):6
4 + 6 -> 10             tek ağaç kaldı (kök 10)
```

Sonuç ağacı, sol 0 ve sağ 1 ile:

```
10
+--0-- 4
|      +--0-- e (2)
|      +--1-- 2
|             +--0-- c (1)
|             +--1-- d (1)
+--1-- 6
       +--0-- a (3)
       +--1-- b (3)

Kodlar: e = 00, c = 010, d = 011, a = 10, b = 11
```

En çok geçen `a` ve `b` ikişer bit, en az geçen `c` ve `d` üçer bit aldı. Hocanın deyişiyle "şans yaver gitti", `e` de iki bitle gösterildi. Toplam $3 \cdot 2 + 3 \cdot 2 + 1 \cdot 3 + 1 \cdot 3 + 2 \cdot 2 = 22$ bittir; normalde $10 \times 8 = 80$ bit tutardı (ara adım eklendi).

Soru: İlk adımda 2'yi ortaya eklememizin bir anlamı var mıydı? Başa da ekleyebilirdik, değil mi?

Cevap: Başa da eklenebilirdi. Hoca araya eklemeyi göstermek için özellikle öyle yaptı. Eşitlikte nereye eklendiği, listedeki karşılaştırmanın nasıl yazıldığına bağlıdır, hiç fark etmez.

> **Ders sonrası not:** Hocanın koşulları söylediği yer altyazıda karışık. Artan sıralı listede kesin kural şudur: Listedeki değer yeni değerden **küçük** olduğu sürece ilerlenirse yeni düğüm eşitlerin önüne (burada başa), **küçük veya eşit** olduğu sürece ilerlenirse eşitlerin arkasına (burada araya) girer. Bu örnekte başa eklenirse kodlar `c = 000`, `d = 001`, `e = 01` olur; toplam yine 22 bittir. Kod sözcükleri değişebilir, toplam bit sayısı değişmez.
>
> **Ek bilgi (derste anlatılmadı):** k farklı sembolde tam $k-1$ birleştirme yapılır. Her birleştirmede en küçük iki eleman çıkar, toplamları eklenir. Sıralı bağlı listede ekleme $O(k)$ olduğundan toplam $O(k^2)$'dir; binary heap ile toplam $O(k \log k)$ olur. Huffman ağacının verdiği kod, önek kodlar arasında toplam bit sayısını en aza indirir.

12. Öncelikli Kuyruğa Dönüş: Hizmet Etme ve Yapı Seçimi

12.1 Huffman Ağacı Heap mi?

Hoca ağaca bakıp her ebeveynin çocuklarından büyük olduğunu gösterdi ve "bu aslında öncelikli kuyruk" dedi. Bir öğrenci farkı sordu.

Soru: Bunu min heap'e benzettim. Farkı sola dayalı olmaması mı? Huffman ağacı sola dayalı değildi.

Cevap: Evet. Heap dediğimiz şey mutlaka sola dayalıdır. Huffman ağacının sola dayalı olması gerekmez, o yalnız bir ağaç yapısıdır. Bu yüzden ona heap ağacı denemez. Huffman ağacında kullanılan yapı öncelikli kuyruk mantığındadır, ama burada normal bir ağaçla sağlanır.

Hoca bir de şunu ekledi: Bazı uygulamalarda heap kullanılıp ara düğümlere boş (null) değer konarak sola dayalılık sağlanıyor. Ama bunu yapmak gerekmez ve kafa karıştırmasın diye üzerinde durmadı.

> **Ders sonrası not:** Hoca, kökte en büyük olduğu için ağacın kendisine "öncelikli kuyruk" dedi; sonra ağacın heap sayılamayacağını kendisi söyledi. Doğrusu: Öncelikli kuyruk, birleştirilecek alt ağaçları tutan **listedir** ve her adımda en küçük iki ağırlık çıkarılır, yani bu bir min öncelikli kuyruktur. Huffman ağacı ise sonuçtur ve bir heap değildir.

12.2 Min Heap'ten Sırayla Hizmet

Hoca toparlarken öncelikli kuyruğu tanımladı: İhtiyaca göre en küçük ya da en büyük elemanın kökte yer aldığı yapıdır. Hizmet ederken en öncelikli eleman kolayca alınır.

Örnek. Min heap `[3, 5, 9, 7, 6]`, $n = 5$.

```
3'e hizmet et, 6 köke gelir, n = 4 : [6, 5, 9, 7]
6'nın çocukları 5 ve 9; küçüğü 5   : [5, 6, 9, 7]
6'nın tek çocuğu 7; 6 < 7, dur

5'e hizmet et, 7 köke gelir, n = 3 : [7, 6, 9]
7'nin çocukları 6 ve 9; küçüğü 6   : [6, 7, 9]

6'ya hizmet et, 9 köke gelir, n = 2 : [9, 7]
9'un tek çocuğu 7; 7 < 9           : [7, 9]

7'ye hizmet et, 9 köke gelir, n = 1 : [9]
```

Heap sort'tan farkı şudur: Orada diziyi sıralıyorduk, elemanları kaybetmemek için kökleri dizinin sonunda saklıyorduk. Burada işi biten kuyruktan ayrılır. Saklamaya gerek yoktur, yalnız eleman sayısı bir azaltılır.

12.3 Hangi Yapı Seçilmeli?

Tablodaki yapıların hepsi öncelikli kuyruk için kullanılabilir. Etkinlik açısından en verimlisi binary heap'tir. Ancak kullanım biliniyorsa başka yapı da iş görebilir. Örneğin çok hızlı hareket yoksa ve gelen elemanlar hep en başa yerleşecekse sıralı liste kullanılır. Yer aramak gerekmediği için çok zarar edilmez.

> **Hocanın vurgusu:** Öncelikli kuyruk bu yapıların herhangi biriyle yapılabilir. Ama genel durumda en verimlisi binary heap'tir.

13. Sınavdaki Algoritmik Soruların Çözüm Mantığı

Hoca algoritmik soruların yalnız mantığını anlattı. Sınavdaki diğer sorular derste çözülmedi.

13.1 Seyrek Matris

Seyrek matris (sparse matrix), elemanlarının çoğu sıfır olan matristir. Hoca "yarısından fazlası sıfır" tanımını verip hemen ekledi: Buradaki yarı, gerçekten $n/2$ demek değildir. 100 elemandan 50'si sıfır olan matris seyrek sayılmaz.

> **Ders sonrası not:** Hoca "100 elemanın varsa genelde 10-15'i sıfır" dedi; büyük olasılıkla "sıfırdan farklı" demek istedi, çünkü cümlenin amacı sıfırların çok fazla olduğunu anlatmaktır.

Gerçek dünyadan bir örnek film öneri sistemleridir. Bu sistemler size en çok benzeyen kişileri bulup onların izlediği, sizin izlemediğiniz filmleri önerir. 10 bin film ve 10 milyon izleyici olsun. 10 milyona 10 binlik bir matris elde edilir, ama sizin izlediğiniz film sayısı örneğin yüzü geçmez. Satırınızın geri kalanı sıfırdır. Bütün kullanıcılar için durum benzer olunca matris seyrek olur. O zaman matrisi tam saklamak yerine yalnız sıfırdan farklı elemanlar saklanır.

Soru. Sıfırdan farklı elemanlar (satır, sütun, değer) bağlı listede veriliyor. Liste oluşturulmayacak, yalnız okunacak. Tamamen sıfır olan sütunları bulun.

Listedeki ilk düğümler şöyledir: `(0, 1, 3)` 0. satır 1. sütunda 3 var demektir. Sonra `(0, 3, 1)` ve `(1, 0, 2)` gelir. Hocanın örneğinde cevap 2. sütundur.

Çözüm. Sütun sayısı kadar bir dizi tutulur. Bağlı listenin her düğümünde sütun değerine bakılır ve dizinin o gözü 1 yapılır. Hoca, `A[sütun]++` yapmanın da olabileceğini, önemli olanın gözün sıfırdan farklı olması olduğunu söyledi. Liste bitince hâlâ sıfır olan gözler, tamamen sıfır olan sütunlardır. Hoca başka çözümlerin de olabileceğini ekledi.

Sözlü anlatılan çözümün koda dökülmüş hali aşağıdadır (ara adım eklendi). `gcc -Wall` ile derlenip hocanın üç düğümüyle denendi.

```c
struct dugum {
    int satir, sutun, deger;
    struct dugum *next;
};
```

```c
void sifirSutunlar(struct dugum *head, int sutunSayisi)
{
    int *A = calloc(sutunSayisi, sizeof(int));
    int i;
    struct dugum *ptr;
    if (A == NULL)
        return;
    for (ptr = head; ptr != NULL; ptr = ptr->next)
        A[ptr->sutun] = 1;
    for (i = 0; i < sutunSayisi; i++)
        if (A[i] == 0)
            printf("%d. sutun tamamen sifir\n", i);
    free(A);
}
```

Soru: Listede yalnız değeri ve sonrakinin adresini tuttum. Matrisi 5×6 yerine 1×30 gibi düşünüp sütunu mod alarak buldum. Olur mu?

Cevap: Değer tek başına saklanamaz, değerin yeri de saklanmalıdır. Satır kişiyi, sütun filmi gösteriyorsa, satır ve sütun tutulmazsa kimin hangi filmi izlediği bilinemez. Mod ile hesaplama fikri mantıkça anlaşılır, ama soru açıkça satır, sütun ve değerin hazır verildiğini söylüyor. Mod almaya gerek yoktur. Verilen yapıdan devam edilmelidir.

13.2 Yazıcı Kuyruğu

Soru. Ağdaki bilgisayarlara hizmet veren bir yazıcıya gelen dosyalar ve sayfa sayıları kuyrukta tutuluyor. İki fonksiyon isteniyor:

- Bir iş bitince sıradaki işe hizmet veren fonksiyon.
- Kullanıcının bekleyen bir isteğini, öteki isteklerin sırasını bozmadan silen fonksiyon. Dosya kuyrukta yoksa uyarı verir.

Örnek. Kuyruk: (11 numaralı iş, 5 sayfa), (4 numaralı iş, 3 sayfa), (2 numaralı iş, 7 sayfa).

Soru, yapıyı serbest bırakıyordu: Bağlı liste de dairesel dizi de olabilirdi. Bir öğrenci aradan silmenin bağlı listede daha kolay olacağını düşündüğünü söyledi. Hoca bu düşünceyi doğru buldu.

- **Hizmet:** Baştaki iş 5 sayfaysa 5 kez dönen boş bir döngü yapılır. `for (i = 0; i < ptr->sayfa; i++)` yeterlidir. Sonra kuyruk başı bir sonraki işi gösterir.
- **Silme:** 4 numaralı iş listede aranır. 11 mi? Hayır. 4 mü? Evet. O zaman 11 artık 2'yi gösterir, 4 de `free` edilir. Geriye `(11, 5)` ve `(2, 7)` kalır.

Sözlü anlatılan çözümün koda dökülmüş hali aşağıdadır (ara adım eklendi). Silinecek iş baştaysa liste başını güncelleme ve hizmet sonrası `free` durumları anlatımda geçmedi, kodun doğru çalışması için eklendi.

```c
struct is {
    int no, sayfa;
    struct is *next;
};

struct is *siradakiIs(struct is *head)
{
    int i;
    struct is *ptr = head;
    if (head == NULL)
        return NULL;
    for (i = 0; i < ptr->sayfa; i++)
        ;                       /* boş döngü: sayfalar basılıyor */
    head = head->next;
    free(ptr);
    return head;
```

```
}

struct is *isSil(struct is *head, int no)
{
    struct is *onceki = NULL, *ptr = head;
    while (ptr != NULL && ptr->no != no)
    {
        onceki = ptr;
        ptr = ptr->next;
    }
    if (ptr == NULL)
    {
        printf("Uyari: %d numarali is kuyrukta yok\n", no);
        return head;
    }
    if (onceki == NULL)
        head = ptr->next;
    else
        onceki->next = ptr->next;
    free(ptr);
    return head;
}
```

13.3 Kuyruğu Yığınla Ters Çevirme

Kuyrukta 1, 2, 3, 4 sayıları var ve bir kod parçası çalıştırılınca kuyruğun son hali soruluyor. Sınav metnindeki kodda bir hata vardı: `dequeue` yazılması gereken yerde `enqueue` yazılmıştı. Hoca sorunun aslını doğru kodla gösterdi:

```
while (!isQueueEmpty(queue))
{
    value = dequeueQueue(queue);
    pushStack(value, stack);
}
while (!isStackEmpty(stack))
{
    value = popStack(stack);
    enqueueQueue(value, queue);
}
```

İlk döngü kuyruk boşalana kadar elemanları çeker ve yığına atar: Yığında alttan üste 1, 2, 3, 4 olur. İkinci döngü yığın boşalana kadar çeker ve kuyruğa ekler: önce 4, sonra 3, 2, 1. Sonuç: Kuyruk `4, 3, 2, 1` olur, yani ters çevrilir. Kod, hazır kuyruk ve yığın fonksiyonlarının basit dizi uygulamalarıyla `gcc -Wall` altında denendi ve bu çıktıyı verdi.

Hoca, böyle bir hata fark edilirse bunun yazılabileceğini de söyledi: Burada `enqueue` deniyor, ama ne ekleneceği söylenmemiş. Bir öğrenci de atamayı görünce kuyruktan çekildiğini düşündüğünü belirtti. Nitekim `value = ...` ataması, çağrının bir değer döndürmesini gerektirir.

13.4 Karmaşıklık Sorusu

Verilen C kodunun en kötü durum karmaşıklığı Big-O ile soruluyor:

```
int fonksiyon(int n) {
    int i, j, k = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 2; j <= n; j *= 2) {
            k += n/2;
        }
    }
    return k;
}
```

İç döngüde j ikiye katlanır, yani $\log n$ tur döner. Dış döngü $n/2$'den n'e birer birer gider. Big-O'da bu $O(n)$'dir. Çarpım:

$$O(n) \cdot O(\log n) = O(n \log n)$$

Kesin tur sayıları, dış döngü için $n - \lfloor n/2 \rfloor + 1$, iç döngü için $\lfloor \log_2 n \rfloor$ olur (ara adım eklendi).

13.5 Sıralama Algoritmasını Tanıma

Soruda en solda özgün bir dizi, yanında da algoritma 1, 2 ve 3'ün kısmen sıraladığı halleri veriliyordu. Hangisinin insertion, selection ve heap sort olduğu soruluyordu. Örnekteki elemanlar kelimelerdi; anlatımda "from", "have", "no", "bin", "that", "good" geçiyor.

Soru: Bu kararlı (stable) ve kararsız sıralama konusundan mıydı?

Cevap: Hayır, yalnız üç sıralama algoritmasını tanımak isteniyordu.

Soru: Algoritma 3'te elemanlar en küçükten başlayıp sıralanmış gibi duruyordu. Bu yüzden insertion dedim. Doğru mu?

Cevap: Hayır, algoritma 3 selection sort'tur. Selection sort en küçüğü bulup başa koyar, sonra ikinci küçüğü, sonra üçüncü küçüğü. Algoritma 1 ve 2'de ilk gözde en küçük eleman yoktur, o yüzden selection sort olamazlar.

Algoritma 1 ise insertion sort'tur. Baştaki bölüm belli bir noktaya kadar kendi içinde sıralıdır, ama bu bölüme henüz sıra gelmemiş elemanlar girmemiştir. Hocanın önerdiği yol, en kolay tanınandan başlamaktır. Selection sort bulunur, şıklar elenir, sonra hangisinin insertion olabileceğine bakılır. Böylece heap sort'la uğraşmaya bile gerek kalmaz.

13.6 Üçlü Min-Heap'ten Silme

Her düğümün en fazla 3 çocuğu olan bir min heap veriliyor: `[3, 13, 4, 91, 49, 29, 98, 32, 30, 15]`. En küçük eleman silinince dizinin son hali soruluyor.

Üçlü ağaçta i. gözün çocukları $3i + 1$, $3i + 2$ ve $3i + 3$. gözlerdedir (ara adım eklendi).

1. Kök 3 ile son eleman 15 yer değiştirir. 15 köke gelir, 3 çıkar, $n = 9$ olur.
2. Kökün çocukları 13, 4 ve 91'dir. En küçüğü 4'tür. $4 < 15$ olduğu için 15 ile 4 yer değiştirir.
3. 15 artık 2. gözdedir. Çocukları 7. ve 8. gözlerdeki 32 ve 30'dur; 9. göz bölgede değildir. En küçüğü 30'dur. $15 < 30$ olduğu için 15 yerinde kalır.

Sonuç: `4 13 15 91 49 29 98 32 30`. Yani 4 kökte, 15 de 4'ün eski yerindedir.

13.7 Uygun Veri Yapısı

Hoca bu sorunun şıklarını da kısaca söyledi:

İş	Veri yapısı	Neden
Derleyicinin fonksiyonlardan geri dönmesi	Yığın (stack)	Hoca "biliyorsunuz" diyerek geçti
Klavyede basılan tuşları ekrana yazma	Kuyruk	İlk basılan tuş ilk yazılmalıdır. Hızlı yazınca editör yetişemez, tuşlar tamponda (buffer) bekler.
Dengeli olmayan ikili ağacı saklama	Bağlı liste	Dizide saklanmaz
Satrançta bir sonraki hamleyi belirleme	Ağaç	Her hamle bir sonrakine bağlıdır: karar ağacı (decision tree)

Dersin Toparlaması

Bu derste heap ağacının iki temel işlemi kod düzeyinde görüldü. Ekleme ilk boş göze yazıp sift up yapmaktır. Kök silme son elemanı köke alıp sift down yapmaktır. Sift down'da önce çocuk sayısı n'ye bakılarak anlaşılır, sonra büyük çocuk ebeveynle karşılaştırılır. İkisi de ağacın yüksekliği kadar, yani $O(\log n)$ sürer. Heap sort bu işlemi n kez tekrarladığı için $O(n \log n)$'dir ve merge sort'tan farklı olarak ek bellek istemez.

Öncelikli kuyruk, sıradan bağımsız olarak en öncelikli elemana hizmet eder. Sıralı ya da sırasız yapılarla da yapılabilir, ama binary heap ekleme ve çıkarmayı $O(\log n)$'de yaparak en verimli yolu sunar. Huffman kodlaması aynı mantığı veri sıkıştırmaya taşır: En küçük iki frekans tekrar tekrar birleştirilir, sonuçta önek kod veren bir ağaç çıkar. Ağaç sola dayalı olmadığı için heap değildir. Quicksort ve merge sort ise ilerleyen derslerde görülecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.